

ANNA auf dem Raspberry Pi 4 einrichten – Schritt fuer Schritt

Diese Anleitung bringt die ANNA-Web-App auf einem Raspberry Pi 4 zum Laufen und zeigt sie am Handy/Laptop an.

Wichtig zum Verstaendnis: `python run.py` startet den **Echtbetrieb**. Die App zeigt dann ausschliesslich, was die Sensoren melden – gruen = frei, rot = belegt – und laesst sich von Hand nicht veraendern. Sind noch keine Sensoren angeschlossen, startet das Backend trotzdem und meldet unuebersehbar, dass keine Sensordaten ankommen. Eine Simulation gibt es nur, wenn man sie ausdruecklich anfordert (Teil E).

Alles ist zum Kopieren gedacht. Zeilen ohne vorangestelltes Zeichen sind Terminal-Befehle. Nach jedem Schritt steht kurz, was passieren soll.

Was du brauchst

- **Raspberry Pi 4** mit installiertem **Raspberry Pi OS (Bookworm)** auf der microSD-Karte, Netzteil.
- Der Pi und dein **Handy/Laptop** haengen im **gleichen WLAN** (oder LAN).
- Zugriff auf das Terminal des Pi – entweder:
 - direkt: Monitor + Tastatur am Pi, oder
 - per **SSH** vom Laptop (siehe Teil A, Schritt 3).
- **Keine** Sensoren, keine Verkabelung noetig.

Falls auf der SD-Karte noch kein Betriebssystem ist: mit dem Programm **Raspberry Pi Imager** (auf einem PC) "Raspberry Pi OS (64-bit)" auf die Karte schreiben, WLAN und SSH dabei gleich mit aktivieren, Karte in den Pi stecken.

Teil A – Pi startklar machen

A1. Pi einschalten

SD-Karte einstecken, Netzteil anschliessen. Der Pi bootet bis zum Desktop bzw. zur Terminal-Anmeldung.

A2. Terminal oeffnen

Am Pi-Desktop oben das schwarze Terminal-Symbol anklicken. Du landest in einer Eingabezeile wie `pi@raspberrypi:~ $`.

A3. (Optional) Vom Laptop per SSH verbinden

Bequemer als Monitor+Tastatur. Am Pi einmalig die IP herausfinden:

```
hostname -I
```

Gibt z. B. `192.168.1.42` aus (die **erste** Zahlenfolge ist die IP). Am Laptop:

```
ssh pi@192.168.1.42
```

Passwort eingeben – du bist im Pi-Terminal. (Falls SSH nicht aktiv ist: am Pi `sudo raspi-config -> Interface Options -> SSH -> aktivieren.`)

A4. Internet pruefen und System aktualisieren

```
ping -c 3 github.com  
sudo apt update
```

`ping` sollte Antworten zeigen (Internet vorhanden). `apt update` laedt die Paketlisten.

Teil B – ANNA installieren (Echtbetrieb)

B1. Werkzeuge installieren

```
sudo apt install -y git python3-venv
```

Installiert `git` (Code holen) und `python3-venv` (isolierte Python-Umgebung).

B2. Den Code holen

```
cd ~  
git clone https://github.com/farisridzal-hftm/anna.git  
cd anna/backend
```

Legt den Ordner `~/anna` an. Wir arbeiten ab jetzt im Ordner `~/anna/backend`.

B3. Python-Umgebung und Abhaengigkeiten

```
python3 -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt  
pip install -r requirements-pi.txt
```

- `python3 -m venv .venv` erstellt die Umgebung.
- `source .venv/bin/activate` schaltet sie ein (die Zeile beginnt danach mit `(.venv)`).
- `requirements.txt` bringt Flask und den Server.
- `requirements-pi.txt` bringt **gpiozero** und **lgpio** – ohne diese beiden kann der Pi keine Sensoren lesen und das Backend startet bewusst gar nicht.

Schlaegt `pip install -r requirements-pi.txt` fehl, hilft meist das Systempaket: `sudo apt install -y python3-lgpio`, danach das venv mit `python3 -m venv --system-site-packages .venv` neu anlegen.

B4. Starten

```
python run.py
```

B5. Startmeldung verstehen

Es sollte in etwa erscheinen:

```
INFO anna.gpio: Feld B1 -> GPIO17 (Header-Pin 11) (pull_up=True, invert=False)
INFO anna.gpio: GPIO-Backend bereit: 7 Sensor(en) aktiv (pin_factory=LGPIIOFactory).
INFO anna: ANNA startet: backend=gpio, areas=2, spaces=7
INFO anna: Web-App erreichbar unter http://0.0.0.0:5000
```

backend=gpio = **Echtbetrieb**. Der Server laeuft jetzt. **Lass dieses Terminal offen**.

Andere Faelle:

Meldung	Bedeutung
Der Echtbetrieb ... benoetigt die Bibliothek gpiozero	Schritt B3 unvollstaendig – requirements-pi.txt nachinstallieren
Nur 3 von 7 Sensoren aktiv	Einige Pins liessen sich nicht oeffnen – siehe docs/Sensor-Inbetriebnahme.md
KEIN einziger GPIO-Pin konnte geoeffnet werden	Meist laeuft noch ein zweiter ANNA-Prozess: sudo systemctl stop anna

Teil C – Web-App oeffnen und testen

C1. Die IP des Pi herausfinden

In einem **zweiten** Terminal (oder kurz mit `hostname -I`, falls du sie nicht mehr weisst):

```
hostname -I
```

Merke dir die erste Adresse, z. B. `192.168.1.42`.

C2. Am Handy/Laptop oeffnen

Im Browser (gleiches WLAN!) aufrufen:

```
http://192.168.1.42:5000
```

(Deine IP statt `192.168.1.42`.) Die ANNA-App erscheint mit den zwei Arealen *Blumenstrasse* und *Hauptstrasse*.

C3. Schnelltest direkt am Pi (optional)

Falls die App am Handy nicht auftaucht, erst am Pi pruefen, ob das Backend antwortet:

```
curl http://localhost:5000/api/health
```

Erwartet im Echtbetrieb:

```
{"status":"ok","mode":"gpio","live":true,"host":"raspberrypi",  
"sensors_ok":7,"sensors_total":7,"sensors_failed":[]}
```

- "live": true → **echte Sensoren**, keine Simulation.
- "status":"degraded" → einzelne Sensoren fehlen (siehe `sensors_failed`).
- "status":"error" → kein Sensor liefert Daten.

C4. Was du siehst

Die Belegung kommt **ausschliesslich von den Sensoren**:

- **Gruene Kachel** = frei, **rote Kachel** = belegt. Ein Auto auf ein Feld stellen → die Kachel wird binnen weniger Sekunden rot.
- Auf eine Kachel **tippen bewirkt nichts** – im Echtbetrieb lässt sich die Belegung nicht von Hand ändern.
- Oben rechts steht ein grünes **LIVE**-Abzeichen. Wäre es Simulation, stünde dort rot **SIMULATION** und darüber ein grosses rotes Warnbanner.
- **Filter** oben (Alle / Normal / Familie / Frauen / Behinderte).
- Beim **+** an einem freien Feld → **Reservierung** (Feld wird gestrichelt, "Reserviert"), **x** hebt sie wieder auf. Das ist eine Vormerkung und verändert die gemessene Belegung nicht.
- Ist ein Areal voll, erscheint der **"voll"-Hinweis** mit Verweis auf das andere Areal. **"Route"** öffnet Google Maps, unten steht die **Auslastung**.

Meldet die App, dass Sensoren fehlen, oder reagiert eine Kachel nicht: `docs/Sensor-Inbetriebnahme.md` und die Diagnose-Seite `http://<IP>:5000/diag`.

C5. Server beenden

Im ersten Terminal **Ctrl + C** drücken. Der Server stoppt.

Teil D – Autostart einrichten (optional, fuer die Vorfuehrung)

Damit die App **automatisch beim Einschalten** des Pi startet. Am einfachsten mit dem mitgelieferten Skript – es legt `venv`, Abhängigkeiten und den Dienst in einem Rutsch an:

```
bash ~/anna/backend/deploy/install.sh
```

Wer es von Hand machen will, kopiert diesen Block **einmal** komplett ins Terminal (er trägt Benutzer und Pfad automatisch ein):

```
sudo tee /etc/systemd/system/anna.service >/dev/null <<EOF  
[Unit]  
Description=ANNA Smart-Parking  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple
```

```

User=$USER
WorkingDirectory=$HOME/anna/backend
Environment=ANNA_BACKEND=gpio
Environment=ANNA_HOST=0.0.0.0
Environment=ANNA_PORT=5000
Environment=ANNA_DIAG=1
ExecStart=$HOME/anna/backend/.venv/bin/python run.py
Restart=on-failure
RestartSec=3

[Install]
WantedBy=multi-user.target
EOF
sudo systemctl daemon-reload
sudo systemctl enable --now anna

```

Achtung bei Handstarts: Laeuft der Dienst, haelt er die GPIO-Pins. Ein zusaetzliches `python run.py` im Terminal bekommt dann keinen Pin mehr. Vorher immer `sudo systemctl stop anna`.

Verwalten:

```

sudo systemctl status anna      # laeuft es?
journalctl -u anna -f           # Live-Logs (mit Ctrl+C verlassen)
sudo systemctl restart anna     # neu starten
sudo systemctl stop anna        # anhalten
sudo systemctl disable anna     # Autostart wieder abschalten

```

Ab jetzt ist die App nach jedem Booten unter `http://<Pi-IP>:5000` erreichbar.

Teil E – Sensoren anschliessen und pruefen

1. Reed-Schalter je Parkfeld zwischen dem passenden **GPIO-Pin und GND** anschliessen. Welches Feld an welchem Pin haengt, steht in `config/parking_layout.json`:

Feld	GPIO (BCM)	Physischer Header-Pin
B1	17	11
B2	27	13
B3	22	15
B4	23	16
H1	24	18
H2	25	22
H3	5	29

Beide Spalten beachten! Wer die Pins auf der Steckerleiste abzaehlt, meint die rechte Spalte. Als *physische* Pins waeren 17 und 25 dagegen 3V3 bzw. GND – dort kommt nie ein Signal an. Passt die

Verdrahtung nicht, in `config/parking_layout.json` `"numbering": "board"` setzen.

2. Prüfen, ob das Signal ankommt – die Diagnose-Seite zeigt den **rohen Pegel**:

`http://<IP-des-Pi>:5000/diag` Auto auf ein Feld stellen und wieder wegnehmen, dabei die Spalte **Wechsel** beobachten. Bleibt sie 0, kommt das Signal nicht am Pi an; zählt sie hoch und Belegt/Frei ist vertauscht, genuegt ein Klick auf `invert`.

Ohne Browser geht es auch am Pi: `python scripts/gpio_check.py # Live-Tabelle` `python scripts/gpio_check.py --scan # findet den verdrahteten Pin`

3. Ausführliche Fehlersuche: `docs/Sensor-Inbetriebnahme.md`.

Hinweis: Ein Reed-Schalter reagiert auf ein **Magnetfeld**, nicht auf blankes Metall – dafür kommt ein kleiner Magnet unter jedes Modellauto (Details im `docs/Architekturkonzept.md`, Abschnitt 4.1).

Nur zum Entwickeln: Simulator

Auf einem Laptop ohne GPIO lässt sich die Oberfläche mit simulierten Daten ansehen. Das ist **ausdrücklich anzufordern** und wird in der App durch ein grosses rotes Banner gekennzeichnet:

```
ANNA_BACKEND=simulated python run.py
```

Teil F – Troubleshooting

- **Handy findet die Seite nicht:** gleiches WLAN wie der Pi? IP mit `hostname -I` gegenprüfen. Erst am Pi testen: `curl http://localhost:5000/api/health`. Antwortet das, liegt es am Netzwerk (WLAN-Trennung/Gastnetz/Firewall), nicht an ANNA.
- **Address already in use / Port 5000 belegt:** anderen Port nehmen, z. B. `ANNA_PORT=8080 python run.py`, dann `http://<Pi-IP>:8080` öffnen.
- **(.venv) fehlt vor der Eingabezeile:** Umgebung nicht aktiv – `source ~/anna/backend/.venv/bin/activate` erneut ausführen.
- **pip install schlägt fehl:** Internet prüfen (`ping -c 3 github.com`), danach `pip install --upgrade pip` und den Install-Befehl wiederholen.
- **Dienst-Logs ansehen (bei Autostart):** `journalctl -u anna -f`.

Anhang – Befehle auf einen Blick

```
# Installieren (einmalig)
cd ~ && git clone https://github.com/farisridzal-hftm/anna.git
cd anna/backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
pip install -r requirements-pi.txt

# Starten (Echtbetrieb - Standard)
python run.py # -> http://<Pi-IP>:5000 ; beenden mit Ctrl+C
```

```
# IP des Pi
hostname -I

# Backend-Selbsttest ("live": true = echte Sensoren)
curl http://localhost:5000/api/health

# Sensoren pruefen
#   Browser: http://<Pi-IP>:5000/diag
python scripts/gpio_check.py
python scripts/gpio_check.py --scan

# Autostart-Dienst verwalten
sudo systemctl status anna
journalctl -u anna -f
sudo systemctl restart anna
sudo systemctl stop anna          # VOR jedem manuellen "python run.py"

# Nur zum Entwickeln ohne Hardware
ANNA_BACKEND=simulated python run.py
```